



UMĚLÁ INTELIGENCE

Přednáška 4

Prohledávání ve složitých prostředích

PROHLEDÁVÁNÍ VE SLOŽITÝCH PROSTŘEDÍCH

V přednášce 3 jsme řešili problémy v plně pozorovatelných, deterministických, statických a známých prostředích. Řešením byla posloupnost akcí - cesta z počátečního do cílového stavu. Nyní omezení na prostředí omezení zmírníme, abychom mohli řešit složitější úlohy.

Budeme řešit tyto problémy:

- hledání cílového stavu, aniž by nás zajímala cesta, jak se k němu dostaneme
- prohledávání pro spojitý stavy
- nedeterministické prostředí
- nepozorovatelné nebo jen částečně pozorovatelné prostředí
- neznámé prostředí

V **nedeterministickém** prostředí agent potřebuje podmíněný plán (nikoli posloupnost akcí) a provádí různé akce v závislosti na tom, co pozoruje - například zastaví, bude-li svítit červená, a pojedí, bude-li svítit zelená.

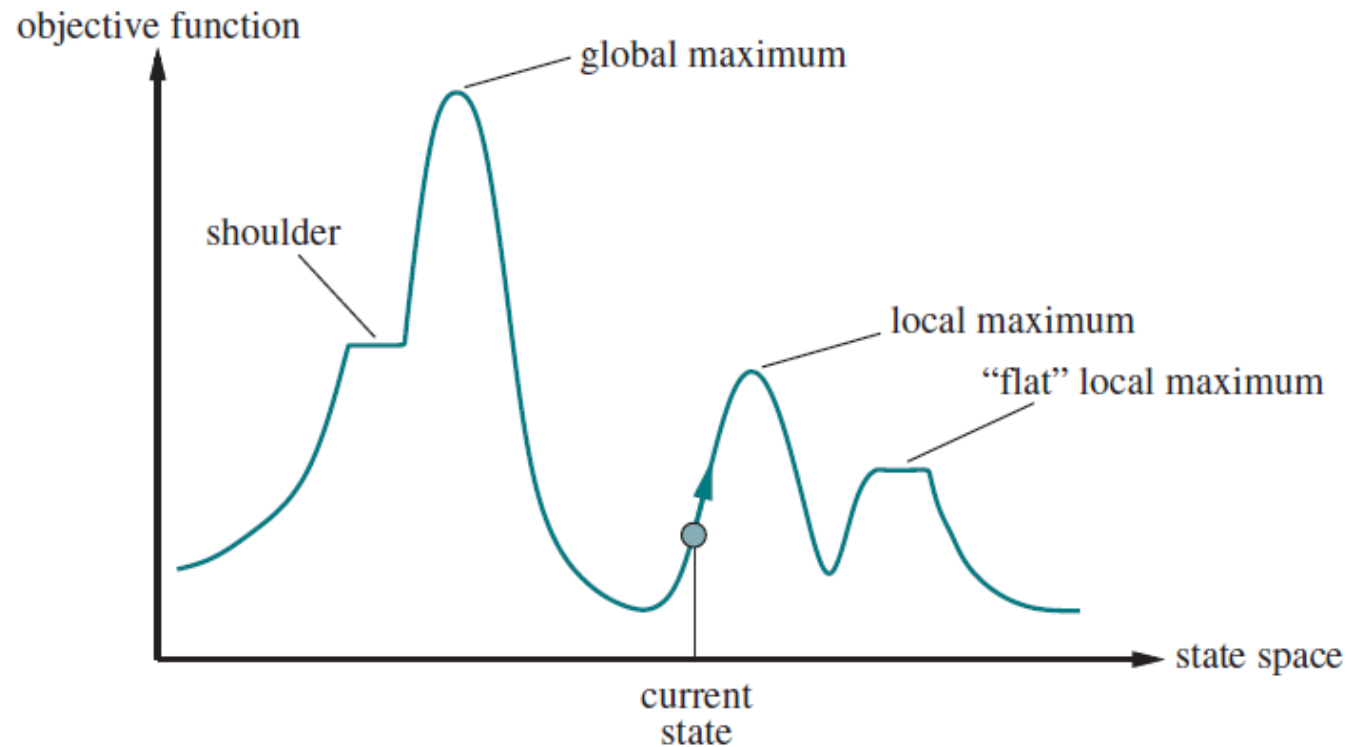
V **částečně pozorovatelném** prostředí agent předpovídá možné stavy, v nichž se může nacházet.

V **neznámém** prostředí se agent učí pomocí online vyhledávání.

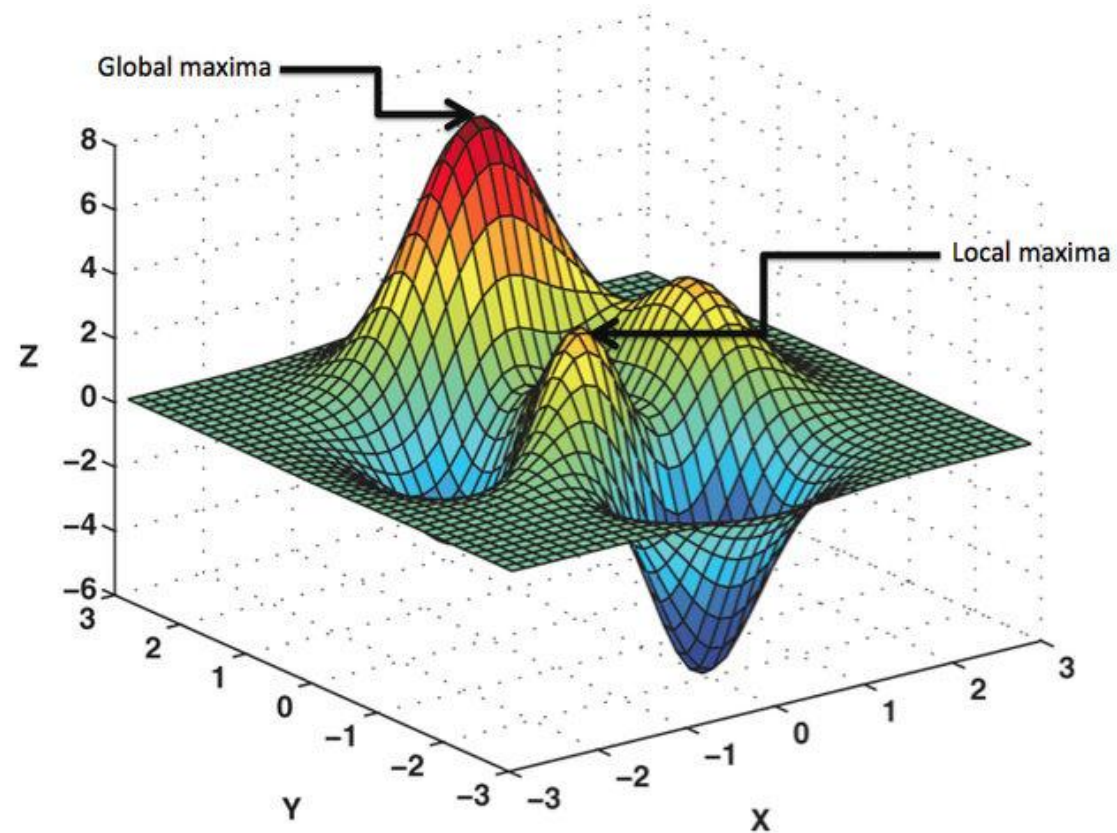
LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

- při lokálním prohledávání agent postupuje od aktuálního stavu k sousedním stavům aniž by ukládal prošlou cestu (tj. nepamatuje si všechny dosažené stavy)
nevýhody - algoritmy nejsou systematické, není zaručeno, že se dostanou do té části stavového prostoru, kde se nachází řešení
výhody - malé nároky na paměť, možnost najít rozumné řešení ve velkých nebo nekonečných stavových prostorech
 - použití pro řešení optimalizačních úloh, tj. nalezení cílového stavu za pomoci **účelové funkce**:
 - každý bod ve stavovém prostoru (stav) má definovanou hodnotu účelové funkce
 - cílem je najít buď "nejvyšší vrchol" — globální maximum nebo "nejhlubší údolí" — globální minimum
 - metody:
 - horolezecký algoritmus/ metoda klesání podle gradientu
 - simulované žíhání
 - lokální paprskové prohledávání
 - genetické algoritmy
-

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY



Příklad účelové funkce v jednorozměrném stavovém prostoru



Globální a lokální optimum ve dvourozměrném stavovém prostoru. Souřadnice X a Y představují hodnoty dvou parametrů a osa Z představuje hodnotu účelové funkce.

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

horolezecký algoritmus – metoda „nejvyššího stoupání“

- udržuje v paměti pouze aktuální stav, každý stav s je ohodnocen pomocí účelové funkce $f(s)$
 - agent se v každém kroku přesune do sousedního stavu s nejvyšší hodnotou účelové funkce
 - konec hledání je na vrcholu, tj. když žádný soused nemá vyšší hodnotu účelové funkce
 - tento algoritmus je hladový – jde rychle k cíli, ale hrozí riziko „zaseknutí“ v lokálním maximu
 - varianty:
 - > **stochastická** - náhodný výběr mezi všemi akcemi, které vedou do stavů s vyšší hodnotou účelové funkce, obvykle pomalejší konvergence, ale lépe najde řešení
 - > **varianta "první volby"** - náhodné generování následníka, dokud nedosáhne lepší hodnoty účelové funkce, než má aktuální stav, strategie vhodná pro stavy s mnoha následníky
 - > **s náhodným restartem** - řada prohledávání z náhodně generovaných počátečních stavů, dokud není dosažen cílový stav - úplný algoritmus, cílový stav může být případně generovaný přímo jako počáteční stav
-

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

horolezecký algoritmus – pseudokód

```
function HILL-CLIMBING(problem) returns a state that is a local maximum  
  current  $\leftarrow$  problem.INITIAL  
  while true do  
    neighbor  $\leftarrow$  a highest-valued successor state of current  
    if VALUE(neighbor)  $\leq$  VALUE(current) then return current  
    current  $\leftarrow$  neighbor
```

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

horolezecký algoritmus – příklad - problém osmi dam

umístit 8 dam na šachovnici tak, aby se navzájem neohrožovaly, tj. stav, kdy žádné dvě nejsou ve stejné řadě, sloupci nebo diagonále

úplné stavové zadání – každý stav obsahuje všechny komponenty řešení (tj. 8 dam), které ale nemusí být správně rozestavěné, v každém sloupci 1 dáma

následníci libovolného stavu jsou všechny možné stavy generované pohybem jedné královny na jiné políčko v daném sloupci (každý stav má $8 \times 7 = 56$ následníků)

heuristická účelová funkce h je počet dvojic královen, které se navzájem ohrožují ($h=0$ pro řešení), za ohrožení se počítá, i když mezi dvěma figurkami stojí další figurka

počáteční stav je libovolný

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

horolezecký algoritmus – příklad - problém osmi dam

- heuristický odhad nákladů aktuálního stavu: $h=17$
- na šachovnici zobrazena hodnota h
pro každého možného následníka získaná přesunem
dámy v rámci jejího sloupce
- existuje 8 tahů, které jsou shodně nejlepší, $h=12$;
algoritmus vybere jeden z nich

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♔	13	16	13	16
♔	14	17	15	♔	14	16	16
17	♔	16	18	15	♔	15	♔
18	14	♔	15	15	14	♔	16
14	14	13	17	12	14	12	18

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

simulované žíhání

- metoda je inspirovaná procesem žíhání v metalurgii, kde se kov zahřeje na vysokou teplotu a pomalu se ochlazuje, aby se atomy uspořádaly do stabilní krystalické mřížky s minimální energií
 - algoritmus dovoluje přechod do horšího stavu s určitou pravděpodobností, což mu umožňuje uniknout z lokálních maxim; tato pravděpodobnost závisí na parametru teploty (T) a velikosti zhoršení (ΔE) - s postupným ochlazováním (snižováním T) se pravděpodobnost „špatných“ kroků snižuje
 - místo nejlepšího stavu je vybrán náhodný stav:
 - pokud je lepší než stávající, tak je akceptován,
 - pokud je horší, tak je akceptován s pravděpodobností $e^{\Delta E / T}$
 - tímto způsobem se algoritmus může dostat z lokálních minim a najít globální minimum
-

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

simulované žíhání - pseudokód

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  current  $\leftarrow$  problem.INITIAL
  for  $t = 1$  to  $\infty$  do
     $T \leftarrow \text{schedule}(t)$ 
    if  $T = 0$  then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}(\text{current}) - \text{VALUE}(\text{next})$ 
    if  $\Delta E > 0$  then current  $\leftarrow$  next
    else current  $\leftarrow$  next only with probability  $e^{-\Delta E/T}$ 
```

Metoda simulovaného žíhání je inspirována procesem žíhání v metalurgii, kdy se materiál zahřeje na vysokou teplotu a pak pomalu ochlazuje, aby se dosáhlo stavu s minimální energií.

V kontextu optimalizace je simulované žíhání metoda hledání globálního minima funkce. Prohledávání začne v náhodně vybraném stavu.

V každém kroku je v okolí aktuálního stavu vygenerován nový stav.

Pokud je nový stav lepší (tj. má nižší hodnotu ztrátové funkce), je akceptován.

Pokud je horší, je přijat s pravděpodobností, která závisí na velikosti zhoršení a na “teplotě” systému.

Teplota je nastavitelný parametr, jehož hodnota se postupně snižuje.

Tímto způsobem se algoritmus může dostat z lokálních minim a najít globální minimum.

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

lokální paprskové prohledávání

- začíná se s k náhodně generovanými stavy, $k > 1$
 - v každém kroku se generuje k následníků - pokud je mezi nimi cíl, algoritmus skončí, v opačném případě se vybere k nejlepších stavů z celkového seznamu
 - výhoda – paralelní hledání, soustředění se na perspektivní směr
 - nevýhoda – riziko malé diverzity – může být zmírněno použitím stochastické varianty, která do výběru přidá prvek náhody, viz dále
-

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

Metoda klesajícího gradientu – gradient descent

- motivace - představte si, že stojíte na kopci a vaším úkolem je najít nejhlubší bod údolí. Máte zavázané oči, ale pod nohama cítíte sklon terénu - uděláte tedy krok směrem, kde terén nejvíce klesá
 - cíl: najít stav, který má minimální hodnotu účelové funkce - zde se tato funkce nazývá ztrátová funkce L (loss function)
 - počítáme gradient ∇ (vektor derivací) ztrátové funkce, který určuje, jakým směrem funkce nejrychleji roste – pro nalezení minima se tedy agent pohybuje proti směru gradientu
 - velikost kroku = parametr učení (learning rate) α
-

LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

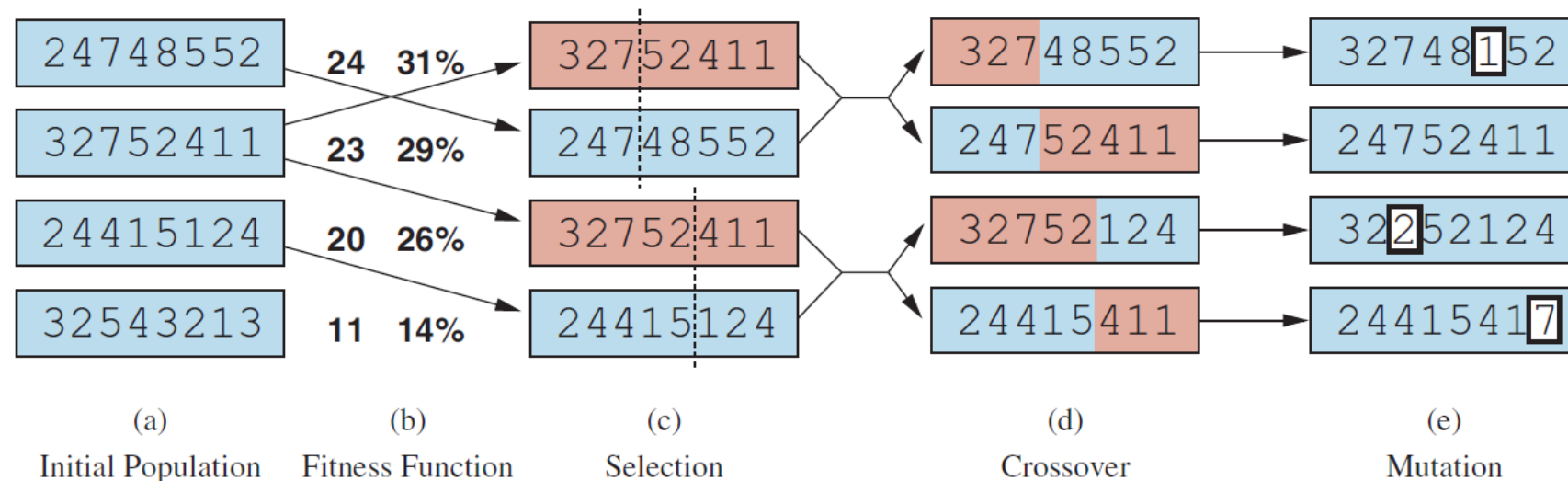
Praktický příklad: Lineární regrese

- agent má odhadnout cenu nemovitosti na základě její rozlohy
 - **model:** $\text{cena} = w \cdot \text{rozloha} + b$
 - **cíl:** Najít parametry w (váha) a b (posun, bias), aby rozdíl mezi odhadem a skutečnou cenou byl co nejmenší.
 - ztrátová funkce L – MSE (střední kvadratická odchylka) – rozdíl mezi skutečnou a očekávanou hodnotou umocněný na druhou a vydělený počtem měření
 - **Postup:**
 - agent začne s náhodnými hodnotami w a b .
 - v každém kroku spočítá L , zjistí gradient ∇L a upravuje w a b , dokud nenajde minimum L :
$$w_{i+1} = w_i - \nabla L(w_i)$$
$$b_{i+1} = b_i - \nabla L(b_i)$$
-

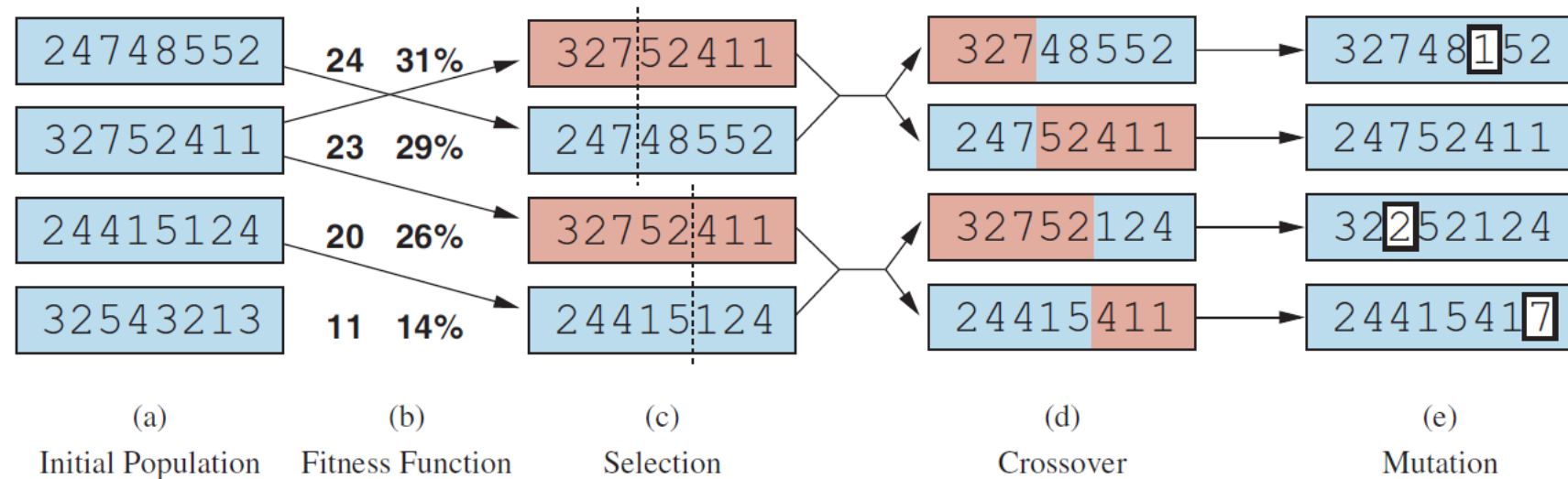
LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY

Genetické algoritmy

- varianty stochastického paprskového hledání, biologická metafora
- populace jedinců (stavy), kdy nejschopnější jedinci (s nejvyšší hodnotou) produkují potomstvo (nástupnické stavy), kteří vytvoří další generaci - tento proces se nazývá rekombinace (křížení)

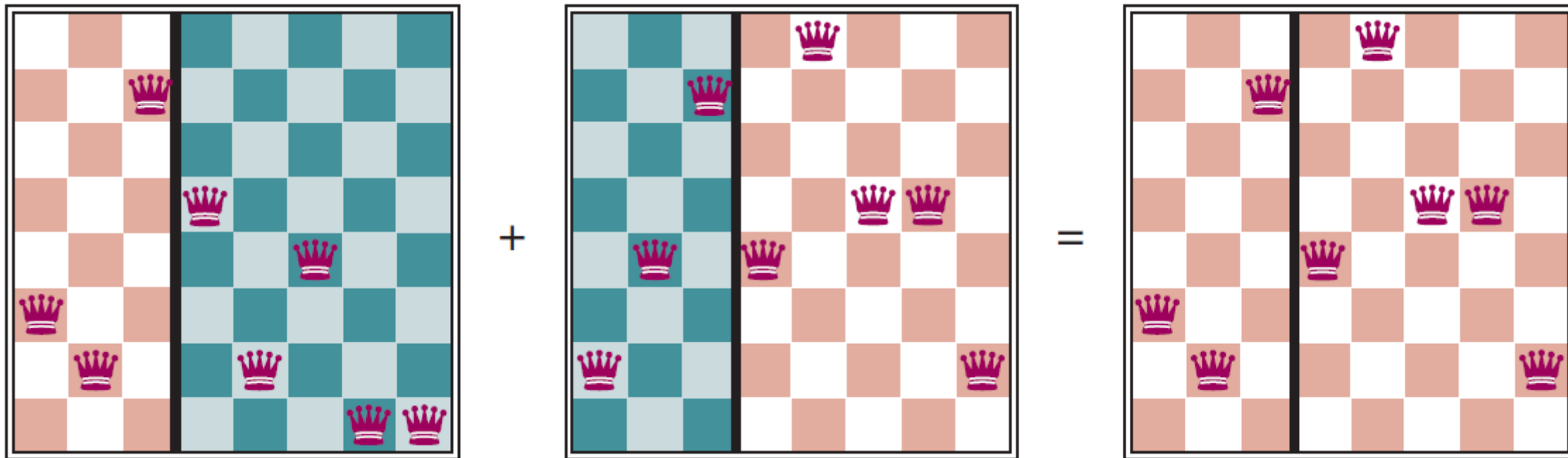


LOKÁLNÍ PROHLEDÁVÁNÍ A OPTIMALIZAČNÍ ÚLOHY



Genetický algoritmus, znázorněný pro řetězce číslic reprezentující stavy 8 dam. Počáteční populace v bodě (a) je ohodnocena fitness funkcí v bodě (b) a výsledkem jsou páry pro křížení v bodě (c). Z nich vzniká potomstvo v (d), které prochází mutací v (e).

Příklad:



```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights  $\leftarrow$  WEIGHTED-BY(population, fitness)
    population2  $\leftarrow$  empty list
    for i = 1 to SIZE(population) do
      parent1, parent2  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child  $\leftarrow$  REPRODUCE(parent1, parent2)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to population2
    population  $\leftarrow$  population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n  $\leftarrow$  LENGTH(parent1)
  c  $\leftarrow$  random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

Genetický algoritmus:

V rámci funkce je populace uspořádaný seznam jedinců,
váhy jsou seznamem odpovídajících hodnot fitness funkce pro každého jedince,
fitness je funkce pro výpočet těchto hodnot.

LOKÁLNÍ PROHLEDÁVÁNÍ VE SPOJITÝCH PROSTORECH

Prostor spojitých akcí má nekonečný faktor větvení, a proto nelze použít předchozí metody.

Způsoby řešení:

- diskretizace – spojitý prostor se rozdělí na konečný počet stavů
 - empirické gradientní metody – výpočet účelové funkce ve dvou blízkých bodech
 - Newton-Raphsonova metoda
 - omezená optimalizace – řešení musí splňovat dané omezující podmínky
 - lineární programování – omezení ve formě lineárních nerovnic
-

V přednášce 3 jsme předpokládali plně pozorovatelné, deterministické a známé prostředí. Agent tedy mohl pozorovat počáteční stav, vypočítat posloupnost akcí, kterými dosáhne cíle, a poté provést tyto akce se "zavřenýma očima", aniž by musel použít své vjemy.

Když je prostředí jen **částečně pozorovatelné**, tak agent neví s jistotou, v jakém stavu se nachází.

Když je prostředí **nedeterministické**, tak agent neví přesně, do jakého stavu přejde po provedení akce.

To znamená, že namísto uvažování "Jsem ve stavu **s1**, a pokud provedu akci **a**, skončím ve stavu **s2**", bude nyní agent přemýšlet "Jsem buď ve stavu **s1**, nebo **s3**, a pokud provedu akci **a**, skončím ve stavu **s2**, **s4** nebo **s5**".

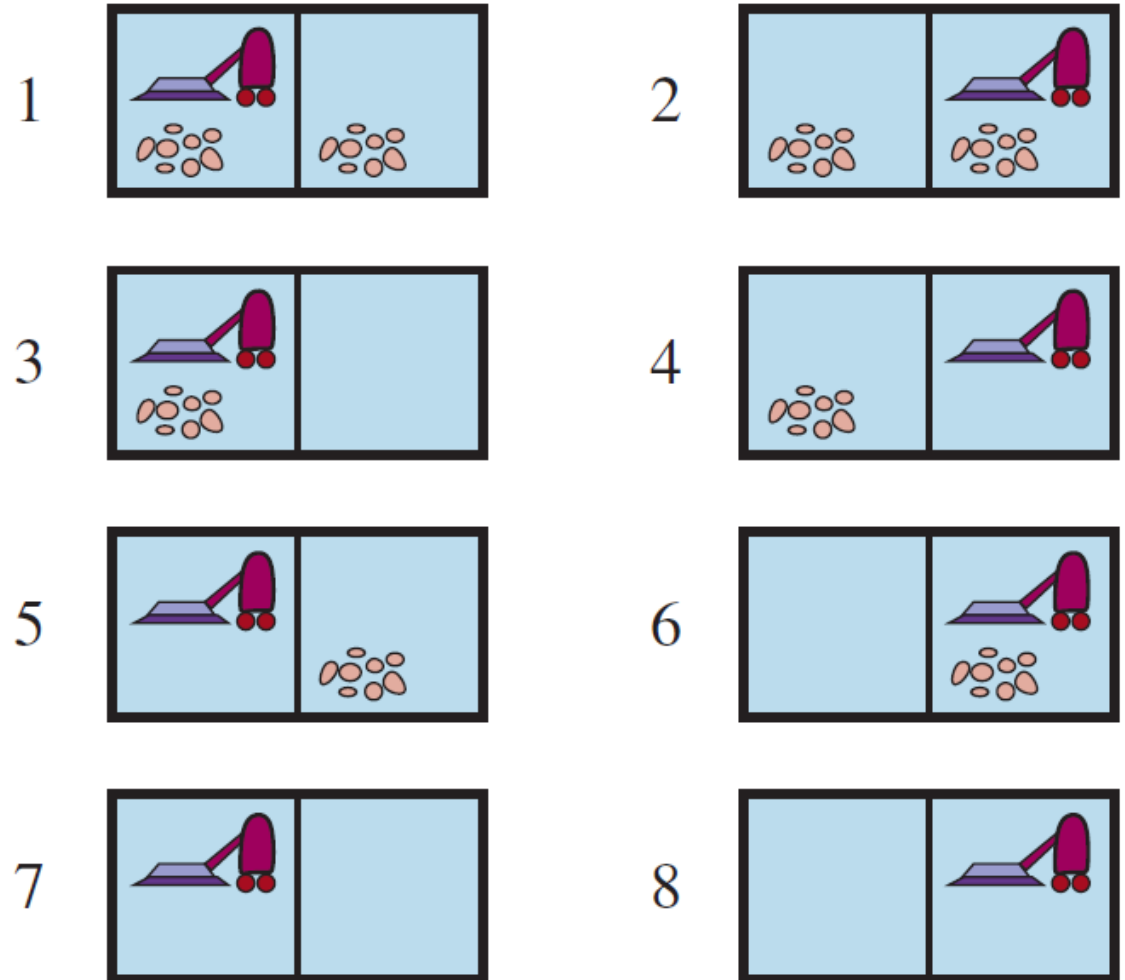
Množinu stavů, o nichž se agent domnívá, že jsou možné, nazýváme **očekávaným stavem (belief state)**.

V částečně pozorovatelných a nedeterministických prostředích již není řešením problému posloupnost, ale **podmíněný plán** (někdy nazývaný **kontingenční plán nebo strategie**), který určuje, co se má dělat v závislosti na tom, jaké vjemy agent při provádění plánu obdrží.

PROHLEDÁVÁNÍ S NEDETERMINISTICKÝMI AKCEMI

nepředvídatelný svět vysavače

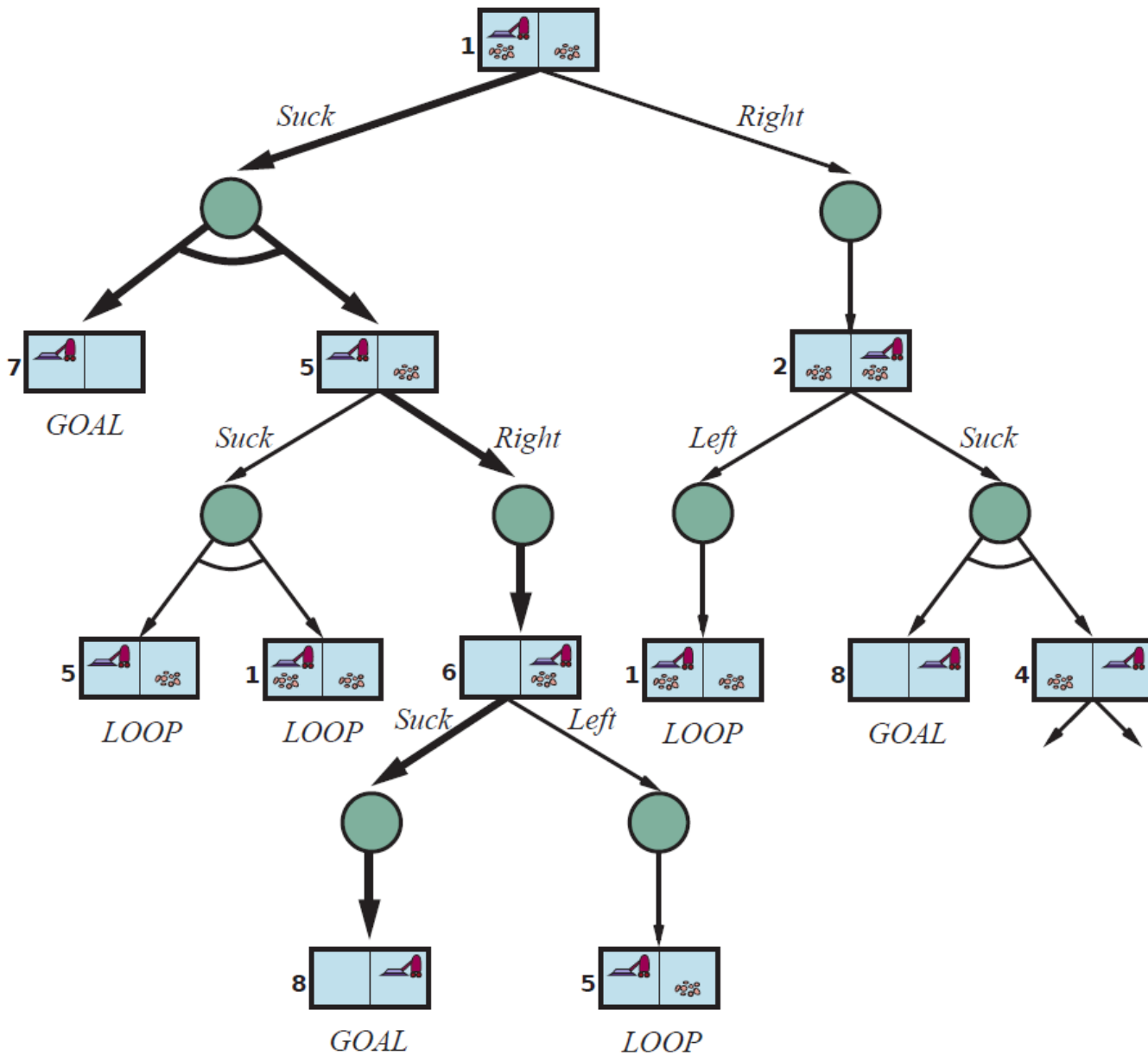
- akce "vysávat" :
 - pokud se použije na špinavé políčko, vysaje smetí, a někdy vysaje i smetí vedle
 - použije-li se na čisté políčko, tak někdy vysype smetí vedle
- zobecnění přechodového modelu z přednášky 3 --> výsledkem akce v aktuálním stavu není nový stav, ale množina možných stavů
- řešením není posloupnost akcí, ale podmíněný plán:
 - strom – **if-then-else** konstrukce



AND-OR prohledávací stromy

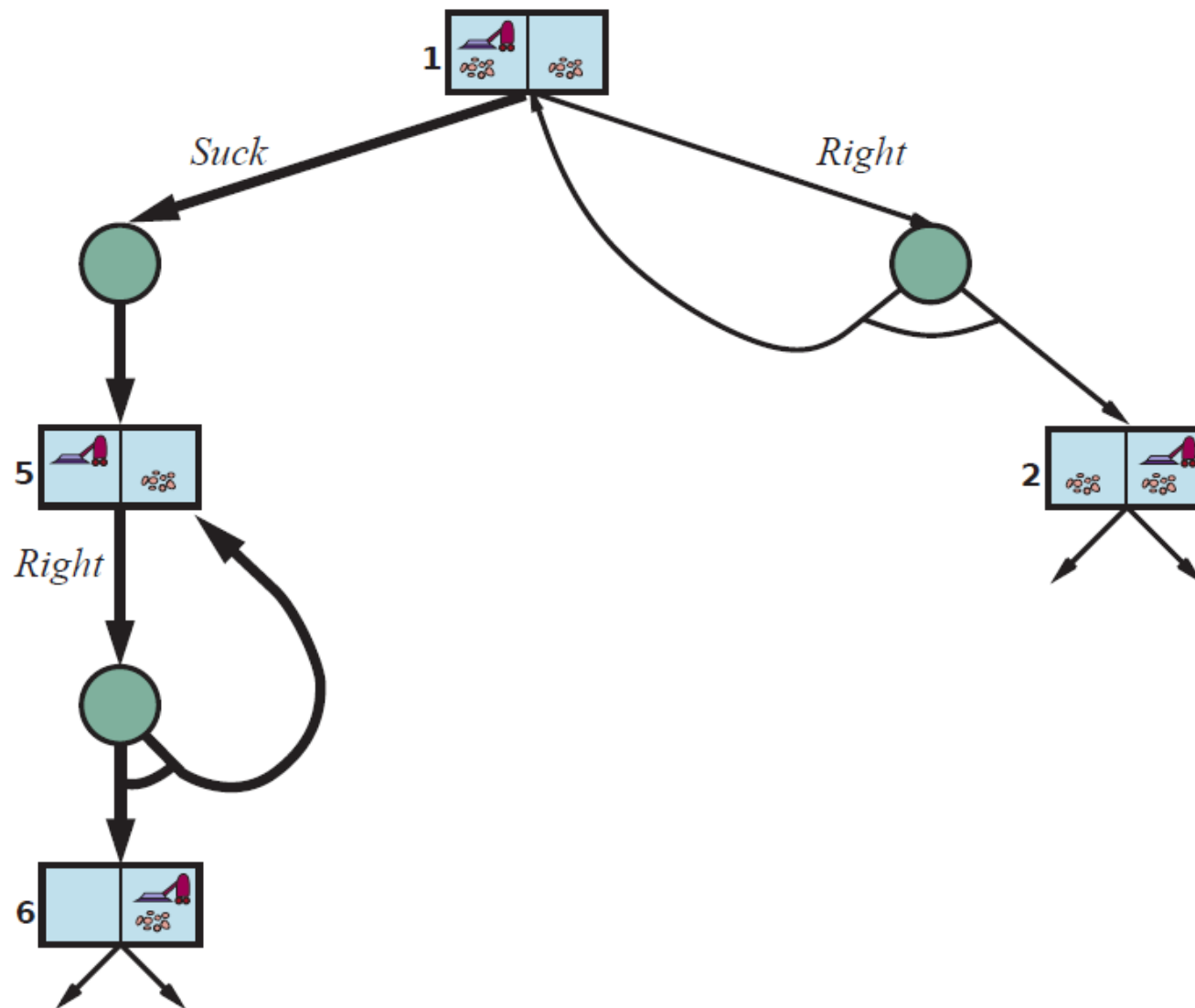
- v deterministickém prostředí je větvení v každém stavu na základě volby akce agenta
→ **OR uzly** grafu
 - v nedeterministickém prostředí vzniká další větvení na základě možných stavů prostředí jako důsledek akce → **AND uzly** grafu
 - tato dvě větvení se pravidelně střídají
 - řešení AND-OR prohledávacího problému je podstrom úplného prohledávacího stromu, který:
 - má cílový stav v každém listu
 - specifikuje jednu akci pro každý uzel **OR**
 - obsahuje všechny výstupní větve uzlů **AND**
 - řešení vyznačeno tučně
-

Příklad:
akce "vysávej" ve stavu 1 způsobí
očekávaný stav {5,7},
tedy agent musí najít plán
pro stav 5 a 7



metoda „zkus, zkus znovu“

- vhodná pro **nejistý svět vysavače**, který odpovídá normálnímu (nikoli nepředvídatelnému) světu vysavače až na to, že akce pohybu je někdy neúspěšná, a tedy agent zůstane na stejném místě



PROHLEDÁVÁNÍ V ČÁSTEČNĚ POZOROVATELNÝCH PROSTŘEDÍCH

prohledávání bez pozorování (pro svět vysavače)

Agent **zná prostředí**, ale neví, kde se nachází, ani jak je rozložené smetí, tedy počáteční očekávaný stav je $\{1,2,3,4,5,6,7,8\}$.

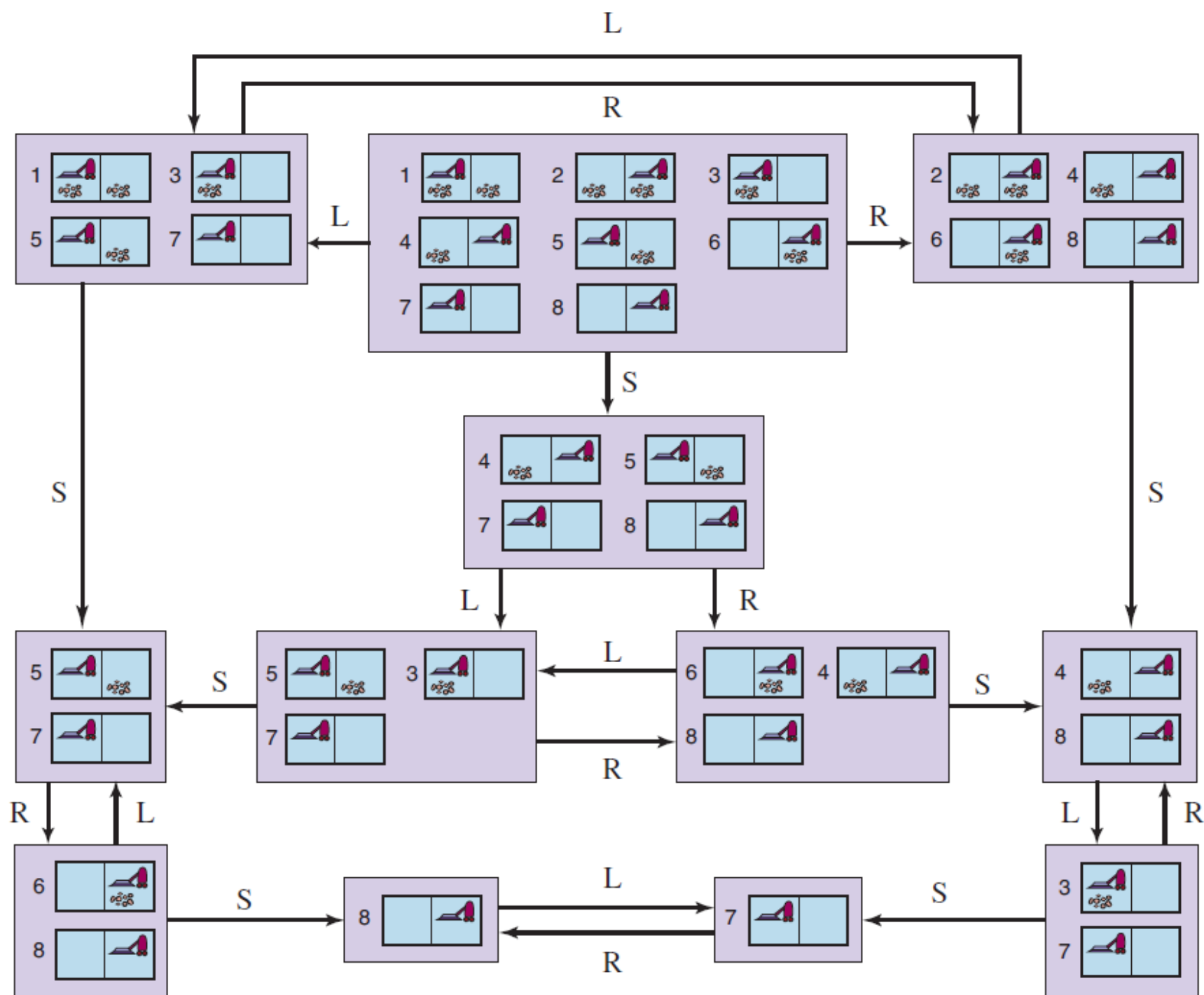
Pokud se nyní pohne vpravo, bude v některém ze stavů $\{2,4,6,8\} \rightarrow$ získal informaci bez pozorování.

Po vykonání posloupnosti akcí [Vpravo, Vysát] agent vždy skončí v jednom ze stavů $\{4,8\}$.

Po sekvenci akcí [Vpravo, Vysát, Vlevo, Vysát] agent zaručeně dosáhne stavu 7, bez ohledu na počáteční stav.

Řešením je posloupnost akcí, hledání probíhá v prostoru očekávaných nikoli skutečných stavů

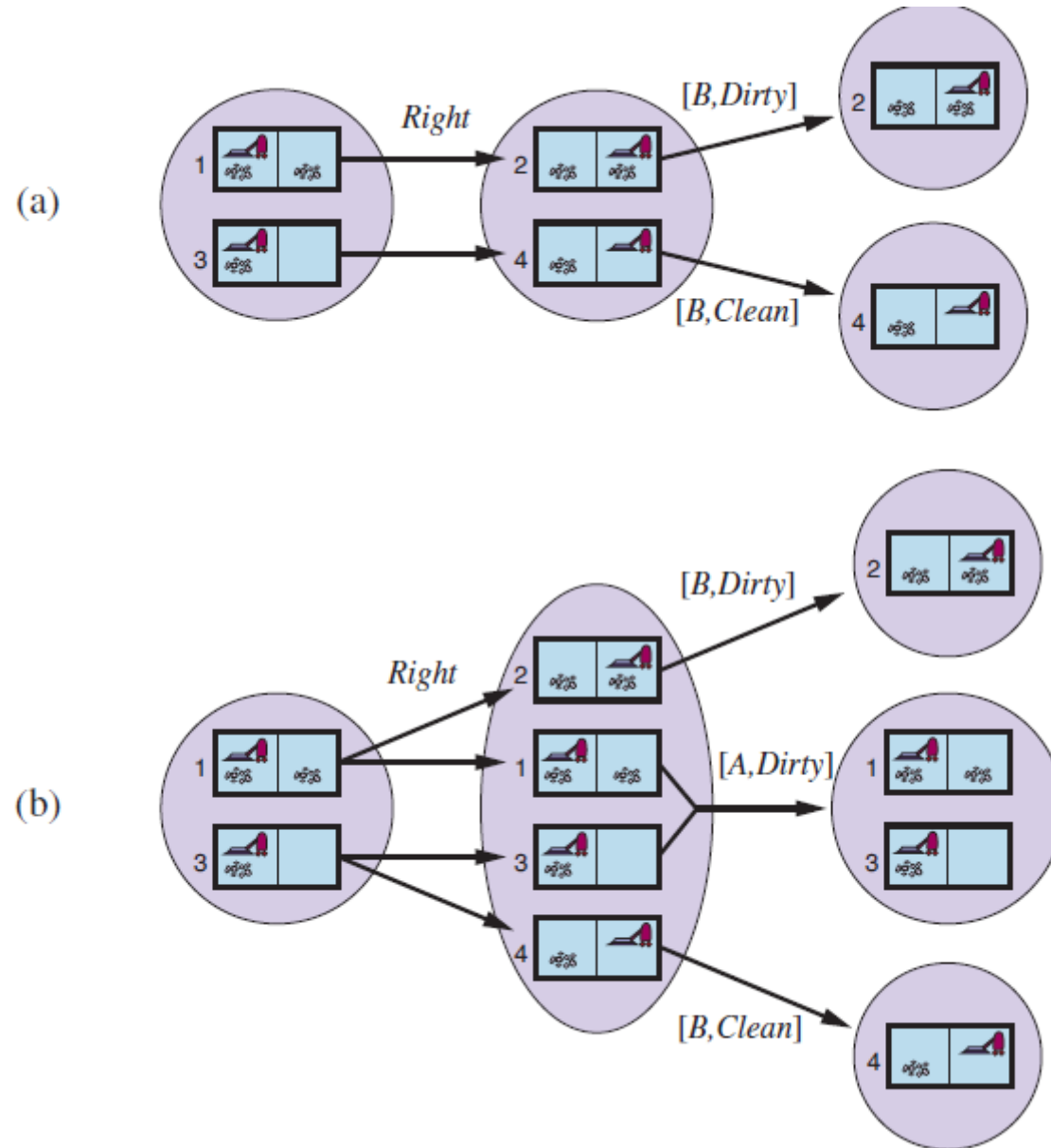
prohledávání
bez pozorování

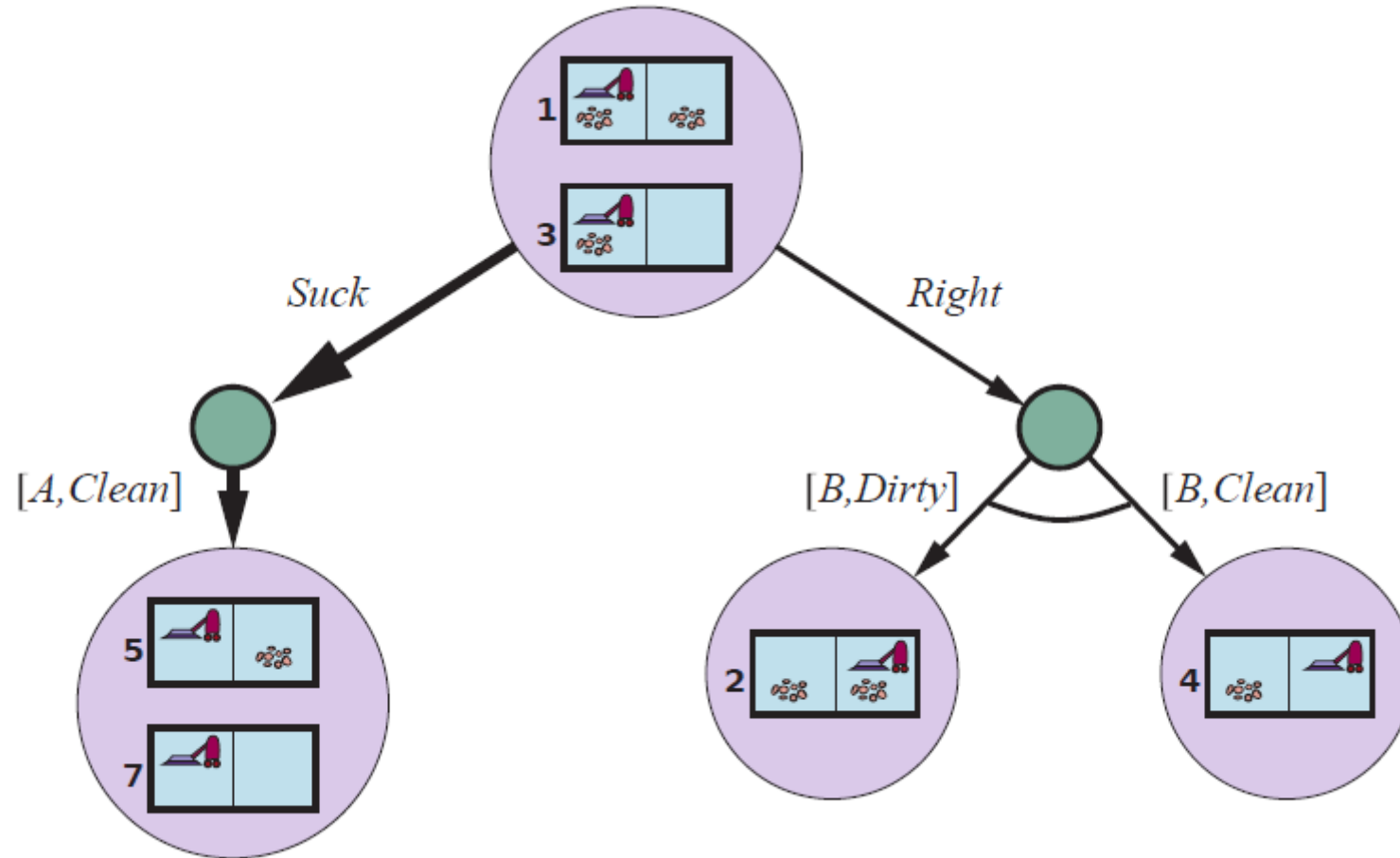


prohledávání v částečně pozorovatelném prostředí

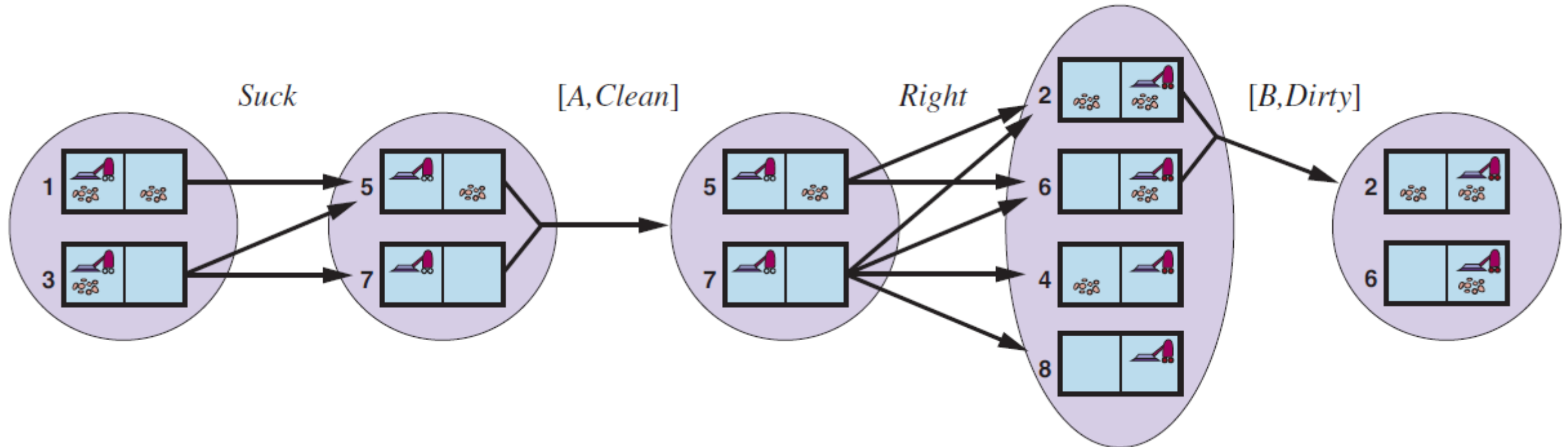
- několik stavů poskytne stejný vjem
- příklad - agent vnímá správně polohu, ale smetí jen v políčku, na kterém se právě nachází, tedy stav 1 i stav 3 poskytnou vjem [L, smetí], tedy očekávaný stav = {1,3}

prohledávání v částečně pozorovatelném prostředí





prohledávání v částečně pozorovatelném prostředí



Shrnutí

Metody lokálního prohledávání uchovávají v paměti jen malý počet stavů. Používají se pro řešení optimalizačních problémů, kdy je cílem nalezení nejlépe ohodnoceného stavu bez ohledu na cestu k tomuto stavu. Většinu těchto metod lze použít pro problémy prohledávání **ve spojitých stavových prostorech**.

Genetický algoritmus je stochastické horolezecké prohledávání, při kterém se uchovává populace stavů. Nové stavy jsou generovány pomocí mutací a křížení, které kombinují dvojice stavů.

V nedeterministických prostředích mohou agenti použít vyhledávání AND-OR ke generování podmíněných (kontingentních) plánů a dosáhnout cíle bez ohledu na to, jaké výstupy nastanou během provádění.

V částečně pozorovatelném prostředí agent místo jednoho stavu uvažuje množinu možných stavů (očekávaný stav - belief state), ve kterých se může nacházet. Standardní prohledávací algoritmy lze aplikovat přímo na prostor očekávaných stavů a řešit tak prohledání bez senzorů.

Zdroje:

Russell, S. J., Norvig, P. *Artificial Intelligence: A Modern Approach*. 4. vyd. Pearson, 2021
